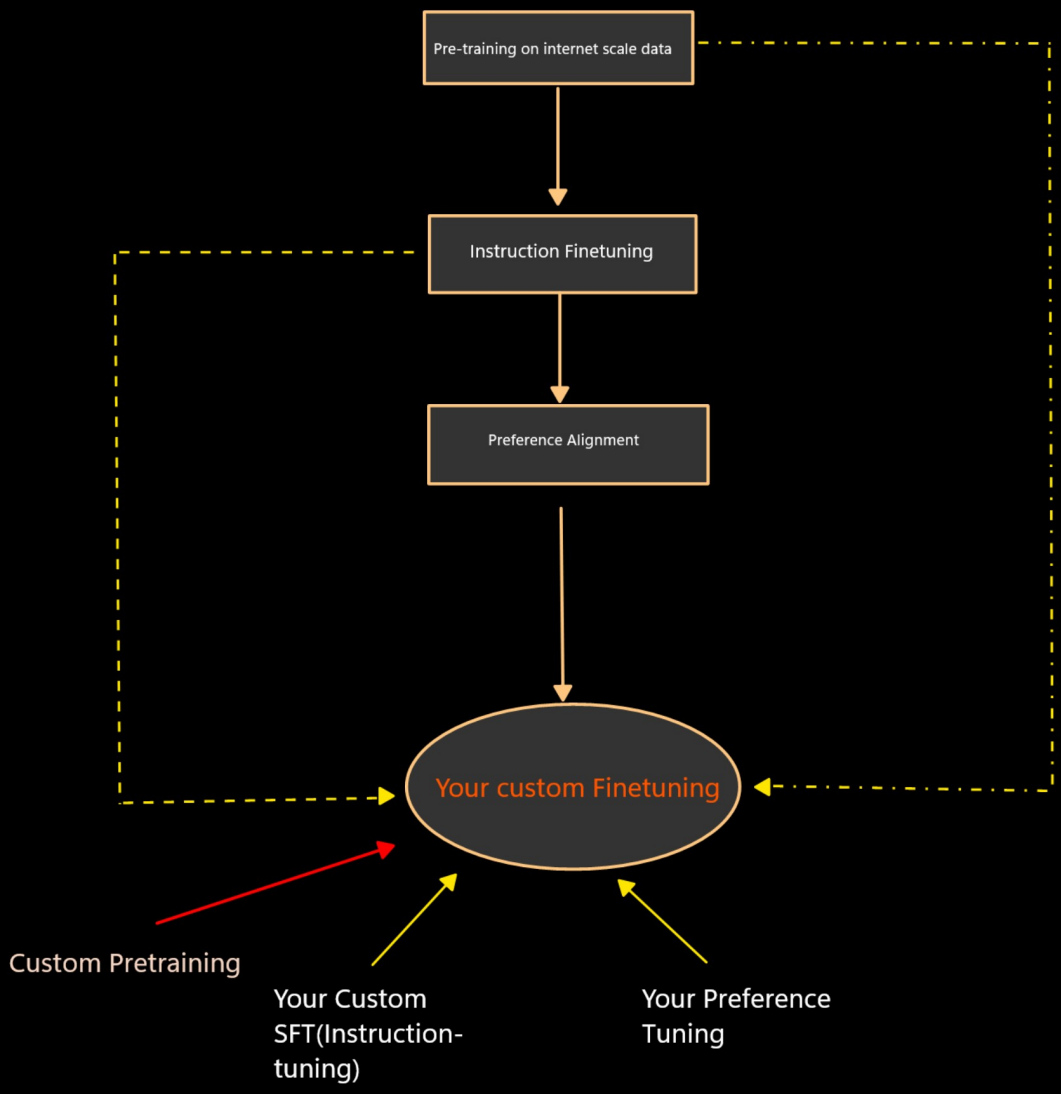
LORA

Understand LLM training
Transfer learning and model fine-tuning
Why fine-tuning was difficult in LSTM models
LSTM vs Transformer
Fine-tuning BERT
LLM knowledge distillation
LLM quantization
LLM fine-tuning with non-instructional data (raw PDF, TXT, documents)
Instruction fine-tuning
Preference alignment
Fine-tuning using tools: Hugging Face, LLaMA Factory, Unsloth, Axolotl
GPT fine-tuning
Gemini fine-tuning
SLM fine-tuning
Multimodal fine-tuning
Embedding fine-tuning

LORA
RLHF(PPO)
GRPO
DPO
ORPO

1. Training stages of LLMs and where this LoRA exist?
2. Full Parameter Fine-Tuning vs PEFT (Parameter Efficient Fine-Tuning) and methods to achieve PEFT
3. What are Weights in Neural Networks and Transformer Architecture?
4. Matrix and Rank in Matrix
5. What is LoRA or LoRA Adapter and benefits of using LoRA
6. What is Optimizer (Weight Update Concept)?
7. Practical of LORA

Understand LLM training



meta-llama/Llama-3.2-1B ← BASE
meta-llama/Llama-3.2-1B-Instruct ← Inst
alignment-handbook/zephyr-7b-dpo-full ← Preference

Own custom model WRT all 3 stage ⇒ Y/N

Full-parameter Fine-tuning
Updates all the model weights
The weight matrices are very large
7B model has 7 billion weights, 13B has 13 billion, and so on
All weights get updated repeatedly for multiple “epochs”

Storing and updating weights requires a lot of memory
which limits fine-tuning to large GPUs or GPU clusters
Expensive Training

Parameter-Efficient Fine-tuning (PEFT)
Does not update all the model weights
Original weight matrices are frozen
Only a small number of additional weights(parameters) are trained

Way of achieve PEFT:
1. LoRA (Low-Rank Adaptation) → QLoRA (Quantized Low-Rank Adaptation)
2. DoRA (Weight-Decomposed Low-Rank Adaptation)
3. Prefix Tuning (adds learnable prefix vectors)
4. Prompt Tuning (learns input embeddings)
5. BitFit (trains only bias parameters)
6. IA^3 (scales attention mechanisms)

Example: LoRA trains small low-rank matrices instead of full weights Training updates only these few parameters repeatedly for multiple epochs
LoRA = Less memory + Faster training + Lower cost + Easy customization
enabling fine-tuning on single GPUs or even consumer hardware
Performance almost same

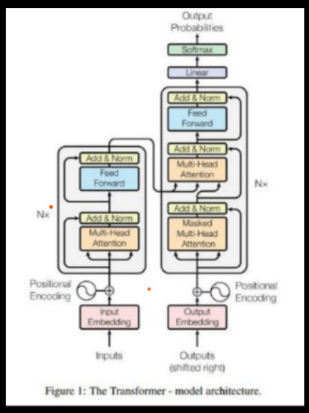
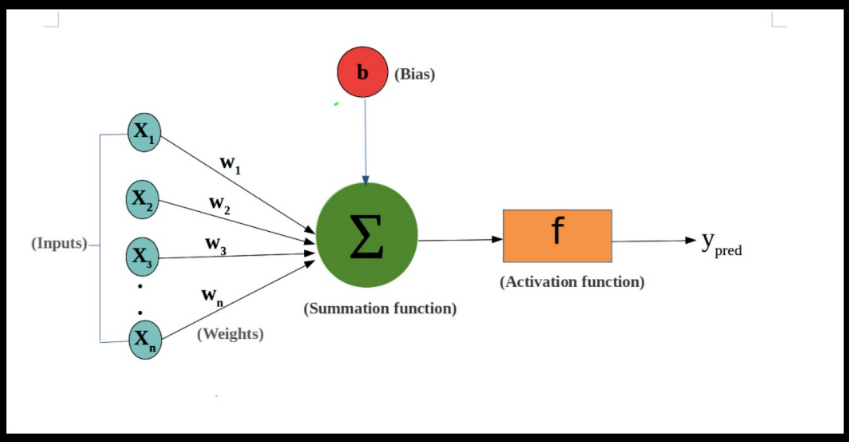
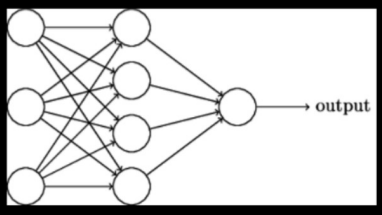


Figure 1: The Transformer - model architecture.

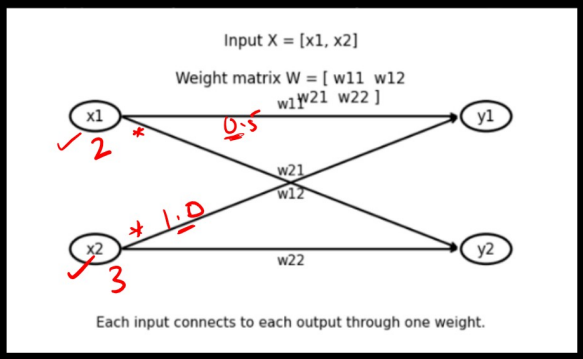
What is weights ?
Weight = importance score
Neural Net: "Which feature is important?"



$$W = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \\ \vdots \\ w_n \end{bmatrix}$$



$$W = \begin{bmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{bmatrix}$$



$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$$

Example:
Input:
x1 = 2, x2 = 3
Weights:
w1 = 0.5, w2 = 1.0

What happens during training?
Initially weights are random
During training:
• Loss is calculated
• *Weights are updated using gradients*
Goal:
Find the best weights

init → randomly
training → adjust → dussl

SF

NN / transformer
FP
Loss
Adjust → weight & bias
BP
optimizer

Imp score

Taste = Patty×0.9 + Cheese×0.7 + Sauce×0.5 + Bun×0.2
w1 w2 w3 w4
I/P I/P I/P I/P

TASTE o/p

Weights in Transformer Architecture

Transformer: "Which word is important for another word?"

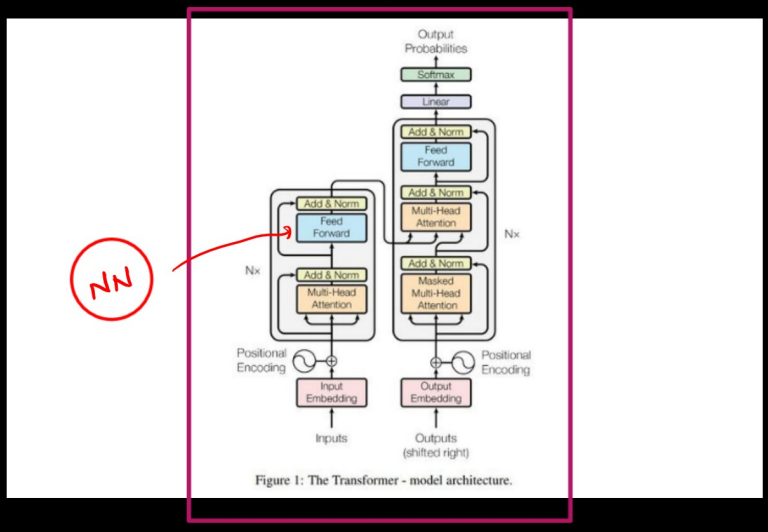
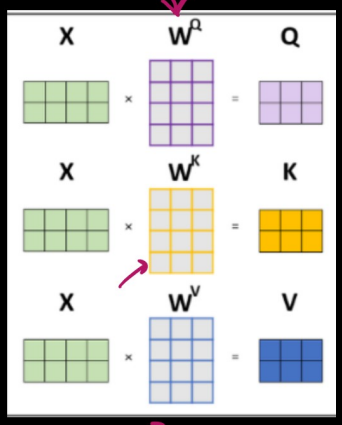


Figure 1: The Transformer - model architecture.

NN

Attention Layer



Linear Layers

- WQ (Query)
- WK (Key)
- WV (Value)

These are all weights matrices

Attention weights

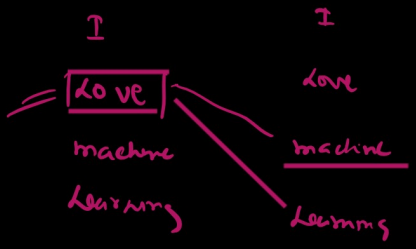
self-attention

transformer

Attention Weights (MOST IMPORTANT)

Model decides:

Which word is important for another word



"I love machine learning"

"learning" focuses on which word?

- love? ✔
- machine? ✔✔

This importance = attention weights

NN & transformer ⟶ Weight

[ ]

Deep Learning = A Matrix Game

Output = Input × Weights + Bias

AI → DL → matrix

Input → vector
Weights(Model parameters) → matrix
Output → transformed vector

Images → matrices (pixels)

Videos → 3D matrices

Text Embeddings → Words → vectors

Attention (Transformers)
Q, K, V → matrices
Attention = matrix multiplication

Matrix and Rank in Matrix

Differences between scaler, vector and matrix

Scalar 5

Vector [1,2,3]

Matrix [[1,2]
[3,4]]

Matrix = a table of numbers (grid)
Matrix = vectors ka collection
Vector = special case of matrix (1 row or 1 column)

A = [1 2
3 4]

This is called a 2×2 matrix.
2 rows
2 columns

More Examples

1. 3×2 Matrix

B = [1 2
      3 4
      5 6]

This is called a 3×2 matrix.
3 rows
2 columns

2. 2×3 Matrix

C = [1 2 3
      4 5 6]

This is called a 2×3 matrix.
2 rows
3 columns

3. m × n Matrix (General Form)

D = [a11 a12 a13 ... a1n
      a21 a22 a23 ... a2n
      ...
      am1 am2 am3 ... amn]

This is called an m × n matrix.
m rows
n columns

Row Matrix

R = [1 2 3 4]

This is called a 1×4 matrix.
1 row
4 columns
Only one row → so it is a row matrix

Column Matrix

C = [1
      2
      3
      4]

This is called a 4×1 matrix.
4 rows
1 column
Only one column → so it is a column matrix

What is Rank?

In simple language: Number of linearly independent rows or columns

Rank = how many independent rows (or columns) are there

How many linearly independent row or column are there?

Real World Analogy
Think:
1000 people
Full rank:
All have different personalities
Low rank:
Only 2–3 types of people, just scaled differently

Visual understanding
Imagine 2D space
Row1 = direction 1
Row2 = direction 2
If both go in different directions → rank = 2
If both go in same direction → rank = 1

General rule
2×2 → max rank 2
3×3 → max rank 3
1000×1000 → max rank 1000

For m×n matrix:
Maximum rank = min(m,n)

- Can one row be formed from another row?
- If yes → information is repeating, Rank is low

Example 1: Full Rank Matrix

A = [1 2
      3 4]

Let's check:
Is second row a multiple of first row?
First row: (1,2)
If we multiply by 3:
(3,6)
But second row is (3,4)
It does not match

Rows are independent
Rank = 2
Full rank

Example 2: Low Rank Matrix

B = [1 2
      2 4]
Second row = 2 × first row
(1,2) × 2 = (2,4)

Second row is not independent
Rank = 1
This is a low rank matrix

Understand from column view
Matrix:

B = [1 2
      2 4]
Columns:
Column1 = (1,2)
Column2 = (2,4)
Column2 = 2 × Column1
Columns are also dependent
So rank = 1

Example 3: 3×3 example

C = [1 2 3
2 4 6
3 6 9]
Notice:
Row2 = 2×Row1
Row3 = 3×Row1
All rows are in the same direction
Rank = 1

Example 4: 3×3 example
Matrix:
D =
[1 2 3
4 5 6
7 8 9]

Reason:
Rows are not fully independent. There is a linear relation:
Row3 = 2 × Row2 - Row1
because
2(4,5,6) − (1,2,3) = (8,10,12) − (1,2,3) = (7,8,9)

This means the 3rd row is dependent.
But:
Row1 and Row2 are independent
2 linearly independent row
Therefore total independent rows = 2
Final Answer: Rank = 2

Rank R or C → Same

Now the question is what to see to check the rank of the matrix? Row or column?
you can check both — the result will be the same

Concept: Rank = number of linearly independent rows = number of linearly independent columns

$$\begin{bmatrix} 1 & 2 & 5 & 3 & 4 \\ 3 & 3 & 9 & 6 & 9 \\ 2 & 3 & 8 & 5 & 7 \\ 4 & 1 & 6 & 5 & 9 \end{bmatrix}$$

4x5 (2)

$C_3 = C_1 + 2C_2$

$C_4 = C_1 + C_2$

$C_5 = 2C_1 + C_2$

What is a Rank of this matrix? ⟶ (2)

how many linear independent row or columns are there? ⟶ (2)

Rank 2

Store Less Parameter

$$\begin{matrix} C_1 & C_2 & C_3 & C_4 & C_5 \end{matrix}$$
$$\begin{bmatrix} 1 & 2 & 5 & 3 & 4 \\ 3 & 3 & 9 & 6 & 9 \\ 2 & 3 & 8 & 5 & 7 \\ 4 & 1 & 6 & 5 & 9 \end{bmatrix} = \begin{matrix} C_1 & C_2 \\ \begin{bmatrix} 1 & 2 \\ 3 & 3 \\ 2 & 3 \\ 4 & 1 \end{bmatrix} \\ A \end{matrix} * \begin{bmatrix} 1 & 0 & 1 & 1 & 2 \\ 0 & 1 & 2 & 1 & 1 \end{bmatrix}$$

B

Actual

4x5 = 20 Parameters

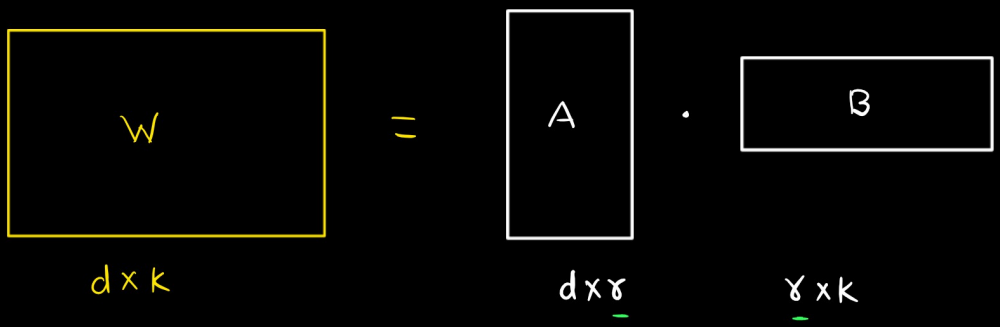
4x2 = 8

2x5 = 10

Rank

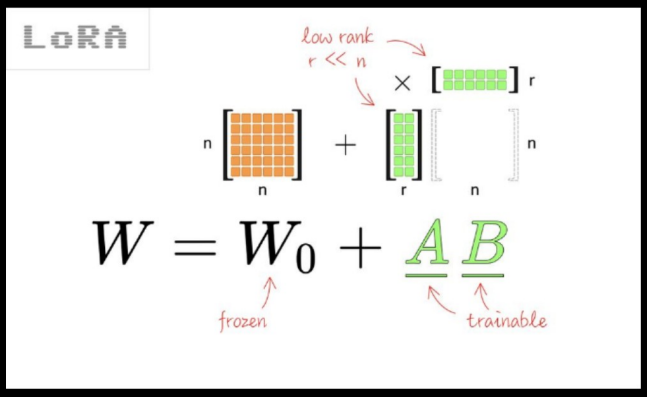
8+10 = 18 Parameters

Low Rank ⟶ which is used in LoRA



Rank

$r < \min[d, k]$



LoRA trains small low-rank matrices instead of full weights

These weight changes are tracked in two separate, smaller matrices that get multiplied together to form a matrix the same size as the model's weight matrix.

LoRA says:
"It is possible that the weight update matrix is not full rank."

Its rank can be small.

LoRA: Learn two small matrices B and A.

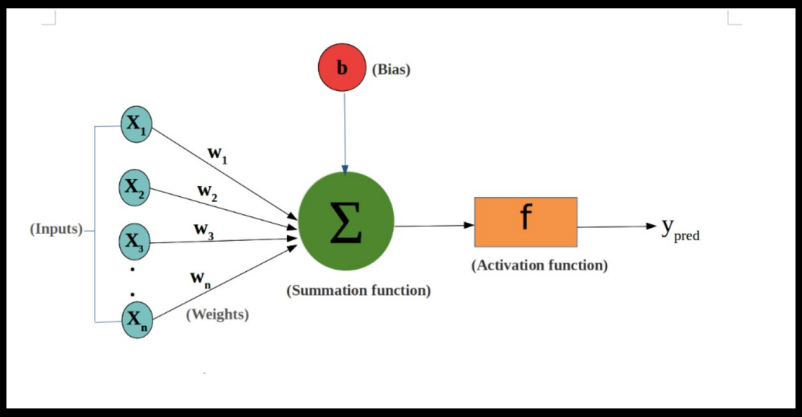
And finally:
$W_{new} = W_0 + BA$

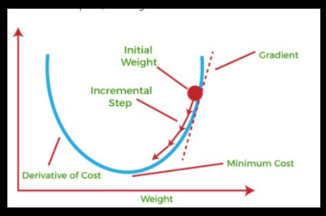
Low-rank update = updating a large matrix by approximating it using smaller matrices.

LoRA

QLoRA(Quantized LoRA)
- This is LoRA 2.0
- Uses even less memory with "recoverable" quantization

Why we should use LoRA?
GPU memory requirements are drastically reduced
Training becomes much faster (even on a single GPU)
Fine-tuning becomes affordable (ideal for startups and individuals)
You can use different LoRA adapters for multiple tasks without duplicating the full model





BP ⟶ Back Propagation

Update weights using Optimizers

GD / SGD: Simple update
GD / SGD with Momentum: Smooth direction
RMSProp: Adaptive learning
Adam: Smart + fastest convergence
AdamW(Weight Decay) + Learning Rate Scheduler + Warmup

GD(Gradient decent)

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W} \longrightarrow L = (y-\hat{y})^2 \longrightarrow \hat{y} = \sigma\left(\sum_{i=1}^{n} w_i x_i\right)$$

$$L = \left[y - \sigma\left(\sum_{i=1}^{n} w_i x_i\right)\right]$$

LORA ⟶ $W = W_0 + \boxed{\Delta W}$

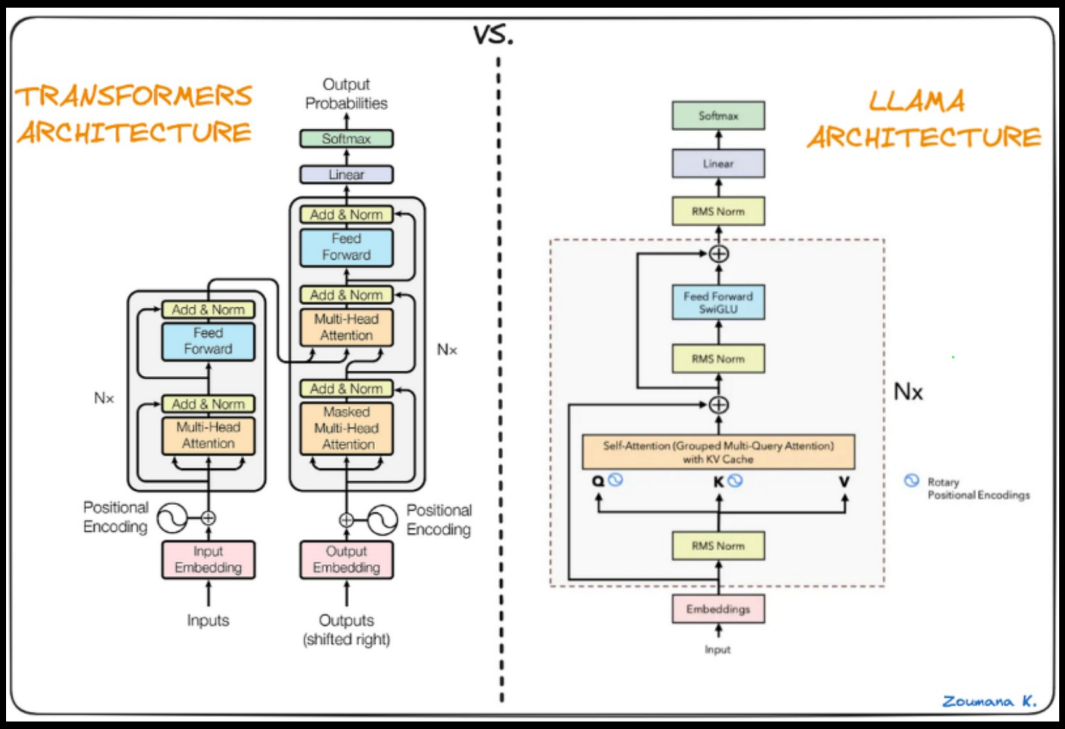
$W = W_0 + BA$

this Parameter (weight or adapter) will be updated

this is adapter or this is weight (Parameter)

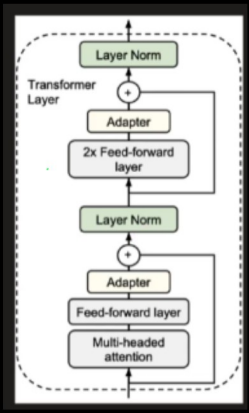
How it will be updated using : AdamW optimizer

Where is LoRA applied in LLMs(Where this adapter is going to be attached)?



What is a Transformer?
Attention + Neural Network
"A neural network architecture built around self-attention"
there:
• Loss is computed
• Gradient is calculated

In Transformer attention, matrices:
• W_q, W_k, W_v, W_o



Paper mostly applies LoRA on:
• W_q and W_v (query & value)

Best performance under same parameter budget:
• W_q alone → average
• W_v alone → average
• But W_q + W_v → best overall

So practical rule:
q_proj + v_proj (most common industry default today too)

Model&Method	# Trainable Parameters	WikiSQL Acc. (%)	MNLI-m Acc. (%)	SAMSum R1/R2/RL
GPT-3 (FT)	175,255.8M	**73.8**	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 ($\text{Adapter}^{\text{H}}$)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 ($\text{Adapter}^{\text{H}}$)	40.1M	73.2	**91.5**	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	**91.7**	**53.8/29.8/45.9**

Feature	LoRA	QLoRA
Model Precision	FP16 / FP32	4-bit (quantized)
Memory Usage	Medium	Very Low
Speed	Fast	Slightly slower
GPU Requirement	16–24GB	8–16GB enough
Library	PEFT	PEFT + bitsandbytes

Type	Bits	Bytes
FP32	32 bit	4 byte
FP16	16 bit	2 byte
4-bit	4 bit	0.5 byte

Quantization is numerical compression of model weights; QLoRA uses 4-bit compressed base weights plus trainable LoRA adapters to reduce GPU memory while keeping fine-tuning effective.

FP32 = 32 bits = 4 bytes
FP16 = 16 bits = 2 bytes
INT8 = 8 bits = 1 byte
INT4 = 4 bits = 0.5 byte

QLoRA combines two ideas:

Quantization + LoRA

Normally FP16 means:

1 parameter = 16 bits = 2 bytes

4-bit means:

1 parameter = 4 bits = 0.5 bytes

Example:

7B model in FP16 ≈ 14 GB
7B model in 4-bit ≈ around 3.5–5 GB with overhead

The flow is:

Install OpenAI SDK
↓
Set API key
↓
Create OpenAI client
↓
Test base model
↓
Load training data
↓
Upload training file
↓
Create fine-tuning job
↓
Check fine-tuned model
↓
Use fine-tuned model for inference

Why Fine-Tuning Is Needed

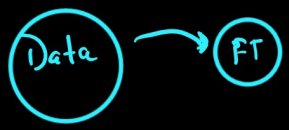
Before fine-tuning, the base model is general-purpose.

It can answer many questions, but it may not follow our company-specific style, rules, tone, or domain behavior consistently.

For example, suppose we are building a smartphone customer support bot.

We want the model to:

Answer only smartphone-related queries
Be friendly and concise
Avoid unrelated topics
Follow company-specific support style



We can try to do this using prompting, but if the behavior must be repeated again and again across many examples, fine-tuning can make the model learn that behavior more consistently.

There are different roles:

system → instruction / behavior setting
user → user question
assistant → model answer
Why is system message needed?

The system message controls the model's behavior.

Example:

You are a helpful assistant. | Medical assistant | Finance assistant

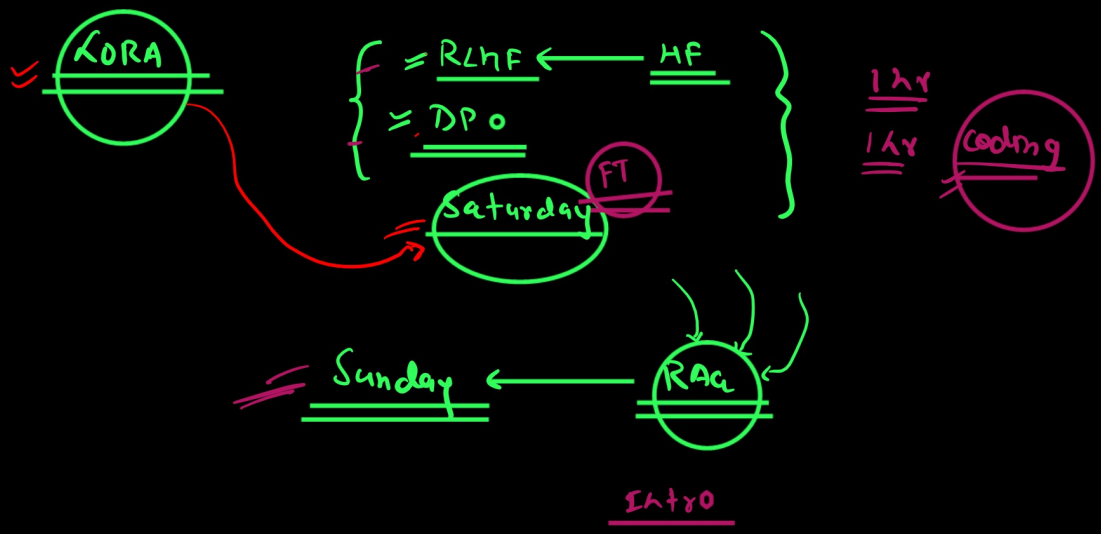
means the model should behave like a helpful assistant.

What happens if we remove system message?

The model can still answer, but we lose behavior control.

{"messages":[{"role":"system","content":"You are a customer support assistant for a smartphone company. You are friendly, concise, and provide only factual answers related to smartphones."},{"role":"user","content":"Can I use the same apps on Android and iPhone?"},{"role":"assistant","content":"Not all apps are available on both platforms. Some apps are exclusive to either Android or iOS, depending on the app store."}]}

General GPT model
↓
Training examples
↓
Fine-tuning job
↓
Custom GPT model
↓
Domain-specific customer support behavior



Intro